

Adalat AI: How It Works

A voice-first moot-court simulator for Pakistani law students. This guide explains the whole system from the beginning, and assumes you know nothing about law and nothing about the code. Every legal term is explained the first time it appears.

WHAT THIS DOCUMENT COVERS

Parts 1 and 2 explain the problem and teach you the courtroom words you need. No technology.

Parts 3 to 9 explain how the software works, one piece at a time.

Parts 10 to 12 cover honesty, measurement, and what is still unfinished.

Part 1

What problem this solves

Law students have to learn how to argue a case out loud, standing up, in front of a judge who interrupts them. Reading about it is not enough. You have to practise it.

The traditional way to practise is a **moot court**. A moot court is a pretend trial. Students argue a made-up case in front of a pretend judge. It is the legal equivalent of a flight simulator.

The problem is people. A moot court needs someone to play the judge, someone to play the lawyer on the other side, and someone to play each witness. Getting four or five people into a room at the same time is difficult, so most students practise far less than they should.

Adalat AI replaces those people with AI. The student speaks out loud into a microphone. An AI judge, an AI opposing lawyer, and AI witnesses answer out loud, each in a different voice. At the end, the student is given marks and written feedback.

The name comes from Urdu: *adalat* means **court**.

THE ONE THING THAT MAKES THIS HARD

A tool that confidently tells a law student the wrong law is worse than no tool at all. A student may repeat it in a real courtroom.

So a large part of this project is not about making the AI sound clever. It is about making the AI provably honest: only citing laws that actually exist, and clearly labelling anything that has not been checked yet. Part 10 covers this.

A crash course in the courtroom

None of the software makes sense until these words do. Read this part even if you skip everything else.

The people in the room

Word	What it means
The Bench	A formal way of saying <i>the judge</i> . The AI plays this role.
Counsel	A lawyer arguing a case. You are counsel . "Learned counsel" is the polite way lawyers address each other.
Opposing counsel	The lawyer on the other side, whose job is to argue against you. The AI plays this role.
Witness	A person who saw or knows something, and answers questions about it under oath. The AI plays these.
Petitioner	The side that brought the case to court.
Respondent	The side defending against the case.

The five phases of a session

A trial runs in a fixed order. This app models five stages, and you move through them one at a time:

#	Phase	What happens
01	Opening	Each side tells the court, in summary, what the case is about and what they want.
02	Examination-in-chief	You question your own witness. Also called "direct examination".
03	Cross-examination	You question the other side's witness, to weaken their evidence.
04	Closing	Each side sums up the whole case and asks for a decision.
05	Verdict	The judge decides. In this app, the judge also marks the student.

The most important rule in the whole app

The difference between phase 02 and phase 03 is the single legal idea the software is built around, so it is worth understanding properly.

When you question **your own** witness, you are **not allowed** to ask a **leading question**. A leading question is one that already contains the answer inside it.

LEADING QUESTION - THE TEST

Leading: "You saw him forge the document, didn't you?"

You have put the answer into the witness's mouth. All they have to do is agree.

Not leading: "What did you see that evening?"

The answer is entirely up to the witness.

The simple test: a question is leading if it states a fact and invites agreement, especially with a tag like "didn't you?", "isn't it?", or "correct?". Questions that begin with **what, where, when, who, why, how** or "describe" are almost never leading, no matter how important the subject is.

Why does this rule exist? Your own witness is on your side. If you were allowed to feed them answers, their evidence would really just be your words coming out of their mouth, and it would be worth nothing. So the law forces you to ask open questions and let the witness speak.

In **cross-examination** the rule flips. There, leading questions **are** allowed, because you are questioning a witness who is not on your side and will not volunteer anything helpful. You are permitted to put propositions to them directly.

Objections

When one lawyer asks an improper question, the other lawyer stands up and interrupts. This is an **objection**. The lawyer says "Objection, My Lord" and then gives a **ground**, which is the specific reason the question is improper.

The judge then does one of two things:

- **Sustains** the objection: "You are right, that question is improper." The witness **does not answer**. The question is struck.
- **Overrules** the objection: "No, the question is fine." The witness answers normally.

The common grounds for an objection:

Ground	Plain meaning
Leading question	The question contains its own answer (explained above).
Hearsay	The witness is repeating what somebody else told them, rather than what they saw themselves. Not allowed, because the person who actually said it is not in court to be questioned about it.
Irrelevant	The question has nothing to do with the issues in the case.
Argumentative or insulting	You are attacking or badgering the witness rather than actually asking them anything.
Assumes unproved facts	The question treats something as already established when it has not been proved.

The actual Pakistani laws involved

Rules of evidence in Pakistan come from a statute called the **Qanun-e-Shahadat Order 1984**, usually shortened to **QSO**. "Qanun-e-Shahadat" simply means "law of evidence". It is divided into numbered

Articles. Three of them carry most of the weight here:

Provision	What it says
QSO Article 136	Defines what a leading question is.
QSO Article 137	You may not ask leading questions when examining your own witness - with some exceptions for introductory or undisputed matters.
QSO Article 138	You may ask leading questions in cross-examination.
QSO Article 71	Oral evidence must be direct. This is the article that makes hearsay objectionable.

Three other statutes are also in the system: the **Pakistan Penal Code 1860** (PPC - which acts are crimes), the **Criminal Procedure Code 1898** (CrPC - how a criminal case is run), and the **Constitution of Pakistan 1973** (fundamental rights).

WHY ARTICLE 137 NEEDS ITS NEIGHBOURS

Article 137 says you cannot lead your own witness - but it has exceptions attached, and the definition of "leading" is in a different article (136), while the permission to lead in cross-examination is in another (138). So you cannot correctly decide a leading-question objection by reading Article 137 alone. You have to read 136, 137 and 138 together. **Remember this - it is exactly why the AI judge in this app was built to look up neighbouring provisions before it rules (Part 6).**

How the software is put together

The system is three separate programs that talk to each other, plus one database. They are separated on purpose, not by accident.

```
YOUR BROWSER (the website you see)
|
| can only talk to its own server. Never talks to
| the AI directly. Never holds the OpenAI password.
v
THE API SERVER (Node.js / Express)
|
| handles: saving to the database, turning your speech
| into text, turning the AI's text back into speech.
| Contains NO AI thinking of its own.
v
THE AI SERVICE (Python)
|
| ALL the thinking happens here: finding the law,
| the judge, opposing counsel, the witnesses, marking.
v
POSTGRESQL DATABASE
```

Why split it up like this?

Because it makes one question easy to answer: **"where does this system make a decision?"** The answer is one folder. All the AI reasoning is in the Python service and nowhere else.

If AI logic were allowed to leak into the API server too, then over time nobody would be able to say with confidence what the system does or why. Keeping them apart, with a network call in between, makes that leak physically inconvenient - which is the point.

There is a second safety benefit. The browser can only talk to its own server, so the OpenAI key never travels to a user's computer. A person using the site cannot steal it, because it was never sent to them.

How the AI finds the right law

Before any AI agent can say anything about the law, it has to find the law. This is the part of the project called **RAG**, which stands for *Retrieval-Augmented Generation*.

RAG IN ONE SENTENCE

Instead of asking the AI to recall the law from memory (where it will confidently invent things), you first **search a real database of law**, then hand the AI the actual text you found and say "answer using only this".

The database holds **53 provisions** - individual sections and articles from the four statutes listed in Part 2.

Two different ways of searching, because there are two kinds of question

People search for law in two completely different ways, and no single search method handles both.

Kind of search	Example	What actually finds it
Word matching	"section 302", "qatl-i-amd"	BM25 - a classic keyword search. It matches the exact words. Very good at exact references.
Meaning matching	"the witness is just repeating what someone else told him"	Vector search - it converts the sentence into a list of numbers representing its meaning, and finds law with a similar meaning. This finds QSO Article 71 even though the two share almost no words.

So the app runs **both searches at once** and combines the results. The combining method is called **Reciprocal Rank Fusion**, and it works on the *positions* the two searches gave (1st, 2nd, 3rd...) rather than their raw scores.

That detail matters. The two searches produce scores on completely different scales - keyword scores have no upper limit, while meaning scores are always squeezed between 0 and 1. Adding those numbers together directly would let the keyword search overwhelm the other one for no good reason. Comparing positions avoids this entirely.

Then a second AI re-reads the shortlist

After combining, an AI model reads the top results and re-orders them by which one actually governs the question. This is called **reranking**.

A RESULT WORTH QUOTING IN YOUR DEFENCE

The obvious choice would have been a small, fast, free, purpose-built reranking model. It was tested against this corpus and it made the results **worse**.

On the query *"the witness is just repeating what somebody else told him outside court"*, the correct answer (QSO Article 71) was ranked:

combined search alone **2nd**
AI reranker **1st**
the small free model **10th**

Why? That small model was trained on web-search results. It rewards shared words and general topic overlap. Nothing in its training connects an everyday description of hearsay to the article that codifies it. A specialist model trained on the wrong subject is worse than no specialist at all.

Overall effect of reranking: the correct provision was ranked first 90% of the time before, and **100% of the time** after.

WHY THERE IS NO VECTOR DATABASE

The usual advice is to store meaning-vectors in a specialised database with a clever index. This project does not, for a good reason: with only **53** provisions, simply comparing the question against every single one is **faster** than any clever index, and it is **exact** rather than approximate.

Those clever indexes exist to approximate an answer when you have millions of records and cannot check them all. At 53 records, you can just check them all.

How the courtroom runs: agents in a graph

This is the heart of the project. The courtroom is not one AI pretending to be several people. It is **several separate AI agents**, each with its own instructions, arranged in a flowchart.

Each agent is a box in the flowchart. The arrows between them are decisions. When you finish speaking, your words enter at the top and travel down whichever path the situation requires.



WHY THIS CANNOT BE DONE WITH ONE PROMPT

One AI told to "play all the characters" cannot reliably produce this sequence: *opposing counsel objects, then the judge looks up the law and rules, and only then does the witness answer - or not.*

That is three agents acting one after another on a single question, where the second one's decision controls whether the third ever speaks at all. Here, that is just a path through the flowchart.

A rule enforced by the structure, not by asking politely

Look at the diagram again. When the judge **sustains** an objection, the arrow goes to **STOP**. There is no arrow from there to the witness.

This means the witness **cannot** answer a struck question. Not because the AI was instructed not to, but because there is no route in the flowchart that would let it. You cannot trick the system into breaking this rule, because you would have to trick it into using a path that does not exist.

This was tested across 32 different scenarios: **zero** cases where a witness answered a struck question.

Everything is bounded

Every loop in the system has a hard limit written into the code. The judge's research loop stops after 3 rounds. The flowchart itself has no cycles at all. An AI system that can loop forever is a system that can run up an unlimited bill, and this one cannot.

How the judge decides an objection

This is probably the most technically interesting part of the project, and the answer to "what does the judge base its decision on?"

The judge is not allowed to answer from memory

A normal AI, asked "is this a leading question under Article 137?", would answer immediately from whatever it happens to remember. That is exactly what we do not want, because what it remembers may be wrong, and it will sound equally confident either way.

Instead the judge is built as a **ReAct agent**. ReAct is short for *Reasoning and Acting*. The agent alternates between thinking and using a tool, in a loop, until it knows enough to answer.

The judge has exactly **one** tool, called `search_statute`, which searches the law database described in Part 4. That tool is the only way it can reach the law at all.

What one ruling actually looks like inside

```
THOUGHT:  This is a leading-question objection under Article 137.
           I need to read 137 together with its neighbours before
           I can rule.

ACTION:    search_statute("leading questions examination-in-chief")

OBSERVE:   QSO 1984 Art. 136 - Leading questions defined
           QSO 1984 Art. 137 - When they must not be asked
           QSO 1984 Art. 138 - When they may be asked

THOUGHT:   137 allows leading on introductory or undisputed matters.
           This question goes to a fact that IS disputed, and we are
           in examination-in-chief. So the exception does not apply.

RULING:    SUSTAINED, citing QSO 1984 Art. 137
```

This is exactly the situation flagged at the end of Part 2: you cannot decide a leading-question objection from Article 137 alone. The judge goes and fetches the neighbouring articles because the law genuinely requires reading them together.

Three things that make this trustworthy rather than decorative

- **It is forbidden from ruling on law it has not read.** The judge's instructions say plainly: rule only on statutory text you have actually read, and do not invent an article that did not appear in what the tool returned.
- **Every step is recorded.** Each thought, each search, and each result is saved. The ruling is not merely grounded in the law - it can be *shown* to be grounded, step by step.
- **The loop is capped at 3 rounds.** Enough to read a provision and its neighbours, and no more. This limits both the waiting time and the cost of a single ruling.

WHAT HAPPENS IF IT FAILS

If the judge uses up its 3 rounds without reaching a decision, the system asks it once more, firmly, with no tools available.

If even that fails, the objection is **overruled** by default. That direction was chosen deliberately: overruling lets a possibly proper question stand, whereas sustaining by default would strike a question that may have been perfectly fine. When you must guess, guess in the direction that does less damage.

How opposing counsel objects on its own

This is the feature that turns a polite question-and-answer session into an actual adversarial hearing.

Nobody presses a button. After every single question the student puts to a witness, the opposing counsel agent independently decides whether that question was improper. If it was, it interrupts.

It physically cannot invent a law

The list of grounds it is allowed to object on is built at runtime from the law database itself. If the AI tries to object on a ground that is not on that list, **the objection is thrown away** rather than shown to the student.

So every objection the student ever hears is backed by a provision that genuinely exists - not because the AI was asked nicely, but because an objection without a real provision cannot survive the code.

The hard part was teaching it to stay quiet

The first version objected too much. Testing caught it objecting "leading question" to this:

THE QUESTION IT WRONGLY OBJECTED TO

"What did you see outside the market that evening?"

This is a perfectly proper open question. It begins with "what" and leaves the entire answer to the witness. The AI appears to have objected because the subject was important, not because the form of the question was wrong.

A lawyer who objects to everything is not a tough opponent - they are just noise, and a student learns nothing from them. Worse, the student might learn a rule that is simply false.

The fix was to write the actual legal test into the AI's instructions explicitly (the test in Part 2: does the question put the answer into the witness's mouth?), including the specific note that questions starting with what, where, when, who, why or how are not leading *however important the subject is*.

Result, measured over 32 labelled test scenarios:

Measure	In plain English	Score
Precision	When it objected, was it right?	1.00 (always)
Recall	Did it catch every improper question?	1.00 (all of them)
Specificity	Did it stay quiet on proper questions?	1.00 (always)
Ground accuracy	Did it name the correct legal reason?	100%

Saving money without lowering quality

The objection check runs after **every** question, and honestly, most questions are fine. Paying for the most expensive AI model to say "no objection" hundreds of times is wasteful.

So the system uses a **cascade**: a cheap, fast model looks at every question first. If it sees nothing wrong, that is the end - cheaply. If it thinks something *is* wrong, the question is passed to the expensive model, which makes the real decision.

This is deliberately lopsided. A missed objection costs the student very little. But **teaching a law student a wrong rule is the expensive failure**, so every objection the student actually hears has been checked by the strong model.

	Before cascade	After cascade
Question with no objection	\$0.0041, 3.5 sec	\$0.0020, 2.3 sec
Question with an objection	\$0.0153	\$0.0153 (1.7 sec slower)
Average per turn	\$0.0104	\$0.0095

Accuracy was identical before and after. All of the saving comes from the quiet questions.

How the AI remembers the session

AI models have a limited working memory. In a long session, the opening argument would scroll out of view long before the closing argument begins. That would be a serious problem here, because catching a student who contradicts themselves is one of the most useful things a courtroom opponent can do.

So the system keeps memory in two layers:

Layer	What it holds	Cost
Working memory	Every exchange in the current phase, word for word.	Free
Long-term memory	A structured summary of everything before it: a narrative summary, a list of claims the student committed to, what each witness said, and any directions the judge gave.	About 4 AI calls per whole session

The long-term memory is kept as a **structured list** rather than a flowing paragraph. That is deliberate: the list of "things the student has claimed" is precisely what a contradiction check needs to compare against. Blending it into prose would make that comparison fuzzy and unreliable.

The summary is only rewritten when the session moves from one phase to the next - not after every single sentence. Within a phase the AI can already see the raw exchanges, so summarising them would be paying money to produce information it is not missing.

WHAT THIS BUYS YOU

Because of this memory, the opposing counsel can say something like: *"My learned friend told this court in his opening that his client was not present at the site. He now says his client witnessed the demolition. Which is it?"*

That is the behaviour that makes the simulator feel like a real opponent rather than a chatbot.

How the student is marked

Marking is a completely separate mechanism from ruling on objections. At the end of a session, three things happen **before** the AI is asked to give any marks.

- **Step 1 - Check every law the student cited.** The system pulls every statutory reference out of what the student said and looks each one up in the database. Each is marked as existing, or not existing.
- **Step 2 - Look up the law that actually governs the case.** This lets the judge tell the difference between a student who *missed* the key provision and one who found it but argued it badly.
- **Step 3 - Hand the judge the results of Step 1 as fact.**

THE INSTRUCTION THAT ANCHORS THE MARKS

The marking AI is told, in effect:

"A citation marked DOES NOT EXIST is a serious failure of legal reasoning, however confident the student sounded, and you must say so explicitly. Do not give a student credit for engaging with a provision they invented."

This is what stops the marks from measuring how *convincing* the student sounded, and makes them measure whether the law they relied on is real.

The student receives five scores out of 100:

Score	What it measures
Legal reasoning	Was the law correctly identified, correctly cited, and correctly applied?
Persuasiveness	Was the argument clear, confident and convincing?
Procedural command	Did the student follow courtroom procedure and etiquette?
Factual command	Did the student know the facts of their own case and the testimony given?
Overall	A weighted combination of the four.

Plus written remarks from the bench, a list of strengths, and a list of areas to improve.

The marks are checked before they are saved

The AI's answer is not simply trusted. If any score is not a proper number, or if the AI names a winning side that is not even a party to this case, the whole verdict is **rejected** and never written to the database. A nonsense verdict never reaches the student.

Does the marking actually discriminate?

This was tested by writing three fake transcripts on purpose - a strong one, a mixed one, and a deliberately bad one - and marking them:

Transcript	Marks given	Citations that were real
Strong performance	85	100%
Mixed performance	55	-
Weak performance	28	0%

The ordering is correct and the gaps are wide. Crucially, the weak transcript's citations were all fake, and the marks reflected that - which shows the judge is reading the citation check, not just reacting to how the writing sounded.

How the system stays honest

This is the part to lead with when explaining the project, because it is the part most similar projects do not have.

THE MOST IMPORTANT DISCLOSURE

All **53** provisions in the database were written from an AI model's knowledge. **They have not yet been checked, word for word, against the official government source** (pakistancode.gov.pk).

So the system marks **every single one** as unverified. This warning appears in the AI's own instructions, and appears on screen beside every citation the student sees.

This is a feature, not an embarrassment. A system that knows and states what it has not checked is more trustworthy than one that presents everything with equal confidence.

Citations are checked before they are spoken, not after

When an AI agent says something during a live session, its citations are checked **before** the words are played as audio. This means if the AI cites something that does not exist, the warning is already on screen while the student is still hearing the sentence.

This is affordable because the check is a simple text-pattern search against a list already held in memory. It costs no money at all, which is what makes it practical to run on every single line.

A real bug this testing caught

The system was attacked on purpose with 36 crafted attempts to make it misbehave. During those tests, the citation checker flagged the **judge** for inventing a law - but the judge had done nothing wrong. The *student* had invented a fake section number, and the judge was correctly naming it in order to **reject** it.

The checker simply could not tell the difference between an AI *relying* on a fake law and an AI *refusing* one. So the code was changed: anything the student already said is tracked separately, and the figure shown to users counts only what the AI introduced by itself.

Result of the attack testing

0 out of 36 attacks succeeded. None of them. Interestingly, attacks aimed at inflating the student's marks made the marks go *down*.

A DECISION NOT TO BUILD SOMETHING

The obvious next step would be to build a defensive filter against such attacks. It was **deliberately not built**.

The reason: nothing got through, so a filter would improve nothing measurable while adding code, cost and latency. It also turned out the system already defends itself for a rather elegant reason - when an attacker hides an instruction inside their spoken argument, the opposing counsel agent **objects to it as irrelevant**.

The rule adopted: build the filter when an attack actually lands, and add that attack to the test set first.

The voice part

The student speaks, and the courtroom speaks back. Each character has a different synthesised voice, so you can tell the judge from opposing counsel from the witness by ear alone.

The waiting problem, and how it was fixed

The first version waited until the entire turn had finished being thought through before playing any audio. Since a single turn can involve opposing counsel objecting *and* the judge researching the law *and* ruling, the student sat in silence for **16.1 seconds**.

The fix was to send each agent's words out **the moment that agent finishes**, instead of waiting for the whole sequence.

	Before	After
Silence before any sound	16.1 sec	8.1 sec
Opposing counsel first heard	-	6.9 sec

The objection is now spoken aloud *while the judge is still reading the statute in the background*. The student hears the interruption at the moment a real opponent would have made it.

Interrupting the court

A real advocate objects *over* the answer, not politely after it has finished. So while the court is speaking, the microphone stays open. If the student starts talking, the audio cuts immediately and whatever they said is sent to the judge as a possible objection.

AN HONEST LIMITATION

The voice system has been proven to work end-to-end using computer-generated speech as the input. It has **never been tested with a real microphone and real speakers**. That test needs a physical device and is still outstanding.

The numbers, and what is left to do

Every claim in this document is backed by a measurement that can be re-run. Here they are together.

What was measured	Result
Finding the right law - before reranking	Correct provision ranked 1st in 90% of test queries
Finding the right law - after reranking	Correct provision ranked 1st in 100% of test queries
Objection decisions (32 scenarios)	Precision 1.00 , recall 1.00 , specificity 1.00
Naming the correct legal ground	100%
Witness answering a struck question	0 occurrences
Marking three planted transcripts	85 / 55 / 28 - correctly ordered
Deliberate attacks that succeeded	0 out of 36
Average cost per spoken turn	\$0.0095
Silence before the first audio	8.1 sec (was 16.1)

TWO NUMBERS TO BE CAREFUL WITH

Two measurements are known to move between runs even when nothing has changed in the code: the judge's *ruling* accuracy has been seen at 94% and at 89%, and the spread of marks on the weak transcript has varied a lot between days.

Never quote either from a single run. Run it three times and quote the average, or say plainly that it is one run. Volunteering this yourself is far stronger than being caught on it.

What is still unfinished

Task	Why it matters	Blocking?
Verify the 53 provisions against the official government source, then switch each one's flag to verified	Until this is done, the system cannot claim to quote real Pakistani law. It is manual reading work and costs nothing.	Yes
Test the voice with a real microphone	Everything else in the voice path is proven; only the physical device path is unconfirmed.	Yes
Move the last two pieces of AI logic out of the API server and into the Python service	Two features still build AI instructions in the wrong place. It works, but it weakens the clean boundary described in Part 3.	No

Task	Why it matters	Blocking?
Operational tooling: request tracing, automated testing on every code change, containerisation, showing cost per session in the app	Cost and speed are already measured per AI call; the rest is polish.	No

Prepared from the Adalat AI source code as it stood on 4 August 2026. All measurements are from the project's own evaluation harness and can be reproduced by re-running it.